

56. Inheritance Definition: What is it and how does it promote reusability?

Inheritance is a mechanism where one class (subclass) acquires the properties and behaviors (fields and methods) of another class (superclass). It promotes code reusability by allowing developers to create a general class once and then create specialized versions of it without rewriting the common code.

57. Superclass vs. Subclass: Differentiate between the two.

- Superclass (Parent): The class whose features are inherited. It represents a more general concept.
- Subclass (Child): The class that inherits from the superclass. It represents a more specific concept and can add its own unique features.

58. extends keyword: What is its purpose?

The extends keyword is used in a class declaration to establish an inheritance relationship. It tells Java that the new class is derived from an existing class (e.g., `class Car extends Vehicle`).

59. super keyword: Explain basic use cases.

The super keyword refers to the immediate parent class.

- Calling Parent Constructor: `super()` is used inside a subclass constructor to invoke the parent's constructor.
- Calling Parent Method: `super.methodName()` is used to call a method from the parent class that might have been overridden in the subclass.

60. Polymorphism Definition: What does "many forms" refer to?

Polymorphism allows objects of different classes to be treated as objects of a common superclass. "Many forms" refers to the ability of a single method name or object reference to behave differently depending on the actual object it is acting upon.

61. Method Overloading: Explain with an example.

Method Overloading occurs when multiple methods in the same class have the same name but different parameters (different type, number, or order).

Example: `add(int a, int b)` and `add(double a, double b)`.

Rule: The method signature (parameters) must be unique.

62. Method Overriding: Explain with an example.

Answer: Method Overriding occurs when a subclass provides a specific implementation for a method that is already defined in its superclass.

Example: A Shape class has `draw()`, and a Circle subclass overrides `draw()` to specifically draw a circle.

Rule: The method name, return type, and parameters must be exactly the same as in the parent class.

63. Compile-time vs. Runtime Polymorphism: Differentiate them.

- Compile-time (Static): Resolved during compilation. Achieved through Method Overloading.
- Runtime (Dynamic): Resolved during execution. Achieved through Method Overriding. It allows a parent class reference to call a child class method at runtime.

64. Abstraction Definition: How does it manage complexity?

Abstraction is the process of hiding the internal implementation details and showing only the essential features of an object. It manages complexity by letting the user focus on what an object does rather than how it does it.

65. Abstract Class vs. Interface: Compare and contrast.

- Abstract Class: Can have both abstract methods (no body) and concrete methods (with body). It can also have constructors and state (variables).
- Interface: Primarily defines a contract. It contains only abstract methods (until Java 8). A class can implement multiple interfaces but extend only one class.

66. Abstract Method: What is it?

An abstract method is a method declared without an implementation (no curly braces). It must be defined in an abstract class. A regular (concrete) class cannot have abstract methods.

67. Interface vs. Abstract Class (More Depth): Concrete methods in interfaces?

Yes, in modern Java (Java 8 and later), interfaces can have concrete methods using the default keyword or the static keyword. This allows developers to add new functionality to interfaces without breaking existing implementations.

68. Basic Inheritance - Animal and Dog

```
class Animal {
    public void makeSound() { System.out.println("Animal makes a sound."); }
}
class Dog extends Animal {
    @Override
    public void makeSound() { System.out.println("Dog barks."); }
}
```

69. Inheritance with Attributes - Vehicle and Car

```
class Vehicle {
    protected String brand;
    protected int year;
```

```

    public Vehicle(String brand, int year) { this.brand = brand; this.year = year; }
    public void displayVehicleInfo() { System.out.println(brand + " " + year); }
}
class Car extends Vehicle {
    private int numberOfDoors;
    public Car(String brand, int year, int doors) {
        super(brand, year);
        this.numberOfDoors = doors;
    }
}

```

70. Method Overloading - Calculator Class

```

class Calculator {
    public int add(int a, int b) { return a + b; }
    public double add(double a, double b) { return a + b; }
    public int add(int a, int b, int c) { return a + b + c; }
}

```

71. Method Overriding - Shape Hierarchy

```

class Shape {
    public void calculateArea() { System.out.println("Generic area calculation"); }
}
class Circle extends Shape {
    double r = 5;
    @Override
    public void calculateArea() { System.out.println("Area: " + (Math.PI * r * r)); }
}

```

72. Runtime Polymorphism - Payment System

```

Payment[] payments = { new CreditCardPayment(), new PayPalPayment() };
for (Payment p : payments) {
    p.processPayment(100.0); // Calls specific version based on object type
}

```

73. Abstract Class - Employee Hierarchy

```

abstract class Employee {
    String name;
    public Employee(String name) { this.name = name; }
    public abstract double calculateSalary();
}

```

```
}  
class FullTimeEmployee extends Employee {  
    public double calculateSalary() { return 5000.0; }  
}
```

74. Interface - Flyable

```
interface Flyable { void fly(); }  
class Bird implements Flyable {  
    public void fly() { System.out.println("Bird flaps wings"); }  
}  
class Airplane implements Flyable {  
    public void fly() { System.out.println("Airplane uses engines"); }  
}
```

75. Combination - Abstraction & Polymorphism

```
abstract class Instrument { public abstract void play(); }  
class Guitar extends Instrument { public void play() { System.out.println("Strumming guitar"); } }  
// ... Main ...  
ArrayList<Instrument> list = new ArrayList<>();  
list.add(new Guitar());  
for (Instrument i : list) { i.play(); }
```